Ines Hoxha

Steven Flores

Report of the Original Dataset

Part 1

The dataset we selected to work with is the Revenue for Cab Drivers. The name of the file is "Revenue for Cab Drivers". It can be found on the link given below where the data set is contained on the 'data.csv' file: https://www.kaggle.com/datasets/diishasiing/revenue-for-cab-drivers The total number of rows is 6,405,008 and the total number of columns is 18.

The attributes contained in the original CSV file are:

- VendorID=>identifies the service provider
- Tpep_ *pickup*_ time=>date and time of pickup
- Tpep_ *dropoff*_ time=>date and time of dropoff
- Passenger_count=>number of passengers in the taxi
- Trip_distance=>distance of the trip in a given measurement unit such as miles or kms
- RatecodeID=> raecde assigned to the trip, represents fare type
- Store_ and_ fwd_ flag=>a Yes/No attribute that indicates whether the trip data was stored locally and then forwarded
- PULocationID=>Pickup Location identifier
- DOLocationID=> Dropoff Location identifier
- Payment_type=> method of payment used by the passenger
- Fare_amount=> base fare for the trip
- Extra=> any additional charges applied during the trip
- MTA_tax=> tax imposed by the MTA
- Tip_amount=>tip given to the service provider, used when applicable
- Tolls_amount=>total amount of tolls charged during the trip
- Improvement_surcharge=>surcharge imposed for improvement of services
- Total_amount=>total fare amount including tips , tax and surcharges
- Congestion_surcharge=>additional charge for trips taken during high traffic congestion times

Keys that can be found in the original dataset are the Vendor ID's, which would specify each specific vendor that provided the service.

Part 2

- Original Relation:
    1. TAXI_TRIP(VendorID, tpep_pickup_datetime, tpep_dropoff_datetime, passenger_count, trip_distance, RatecodeID, store_and_fwd_flag, PULocationID, DOLocationID, payment_type, fare_amount, extra, mta_tax, tip_amount, tolls_amount, improvement_surcharge, total_amount, congestion_surcharge)
    2. No primary key exists. No single attribute is unique because many trips begin in the same second. The most specific combination of attributes, (VendorID, tpep_pickup_datetime, tpep_dropoff_datetime, PULocationID, DOLocationID), has only
    3. Because the relation has no key, no stored attribute can be relied on to determine the others. For example, the pickup time does not determine the drop-off time, and the new pickup location does not determine fare or distance. Every attribute in a row depends on the trip it describes, but no attribute in the dataset identifies a trip.
    4. The original relation has several problems. There are several rows that are identical copies of another row, and without the presence of keys there could be separate trips or the same trip recorded twice.
    5. Two changes must occur to bring the relation into 1NF First, we need a surrogate identifier we call “trip_id”, must be added to every row so that each trip, including duplicates, can be told apart. Next, the repeating group must be removed by splitting the pickup and drop-off columns into separate rows. Each now row will contain a new attribute, which we call “stop_type,” , together with a single stop_datetime and a single location_id.
    6. TAXI_TRIP satisfies three of the rules: the values are atomic, column names are unique, and the ordere in which its rows are stored does not matter. But, what disqualifies it from 1NF is a repeating group, and its rows are not guaranteed to be unique.
- 1NF
    1. We add a surrogate key, trip_id. The repeating pickup and drop-off columns are then split into separate rows, so each trip appears as two rows, one for its pickup and one for its drop-off. We end up with: TRIP_1NF(trip_id, stop_type, stop_datetime, location_id, VendorID, passenger_count, trip_distance, RatecodeID, store_and_fwd_flag, payment_type, fare_amount, extra, mta_tax, tip_amount, tolls_amount, improvement_surcharge, total_amount, congestion_surcharge)
    2. After the split, trip_id alone is no longer a primary key, because every trip_id appears in two row. The primary key is now the composite key

(trip_id, stop_type) Because every row of the original csv file receives its own trip_id, the rows that were duplicates are now distinct.

3. There are three functional dependencies: a combination of trip_id and stop_type determines stop_datetime and location_id, because the time and place of a stop depend on both the trip and whether the stop is the pickup or the drop-off; next, trip_id alone determines the VendorID, passenger_count, trip_distance, RatecodeID, store_and_fwd_flag, payment_type, fare_amount, extra, mta_tax, tip_amount, tolls_amount, improvement_surcharge, total_amount, congestion_surcharge, because attributes describe the trip as a whole and are identical on its pickp and drop-off rows; and lastly, each code determines its own description: VendorID determines the vendor name, RatecodeID determines the rate description, and payment_type determines the payment description.
4. The second functional dependency is a partial one, because fourteen nonkey attributes depend on only part of the primary key, trip_id, rather than on the whole key. As a result, every trip-level value is stored twice, once on the pickup row and once on the drop-off now. The redundancy leads to update, insertion, and deletion anomalies.
5. To reach 2NF, the partial dependency must be removed. The attributes that depend only on trip_id must be moved into their own table with trip_id as its primary key. Also, stop_datetime and location_id remain in a table with primary keys “trip_id”, “stop_type,” with trip_id serving as a foreign key.
6. This is 1NF because it satisfies all five rules: every attribute holds a single atomic value, every column has a unique name, and the order of rows does not affect their meaning. No repeating groups are found. Every row is unique

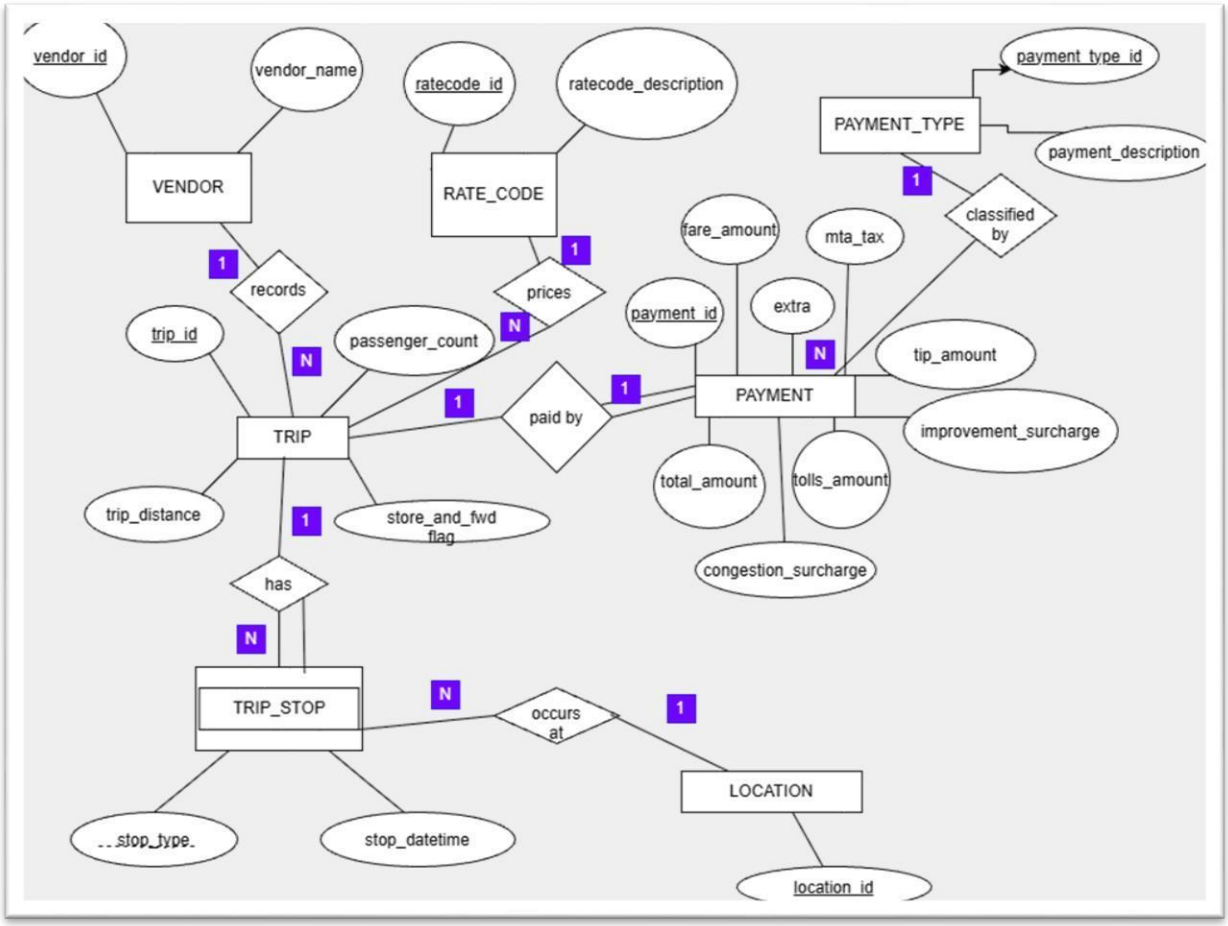
- 2NF

1. The 1NF relation will be divided into seven tables:
    a. VENDOR(vendor_id, vendor_name)
        i. Stores the name of each vendor that records trips
    b. RATE_CODE(rateode_id, rate_description)
    c. PAYMENT_TYPE(payment_type_id, payment_description)
    d. LOCATION(location_id)
    e. TRIP(trip_id, vendor_id, ratecode_id, passenger_count, trip_distance, store_and_fwd_flag)
        i. This table stores the facts about the trip itself, i.e, the rate type, the number of passengers, the distances, and store and forward flag
    f. TRIP_STOP(trip_id, stop_type, stop_datetime, location_id)
        i. This table stores each trip’s pickup and dropoff as two rows, each with their own time and location
    g. PAYMENT(payment_id, trip_id, payment_type_id, fare_amount, extra, mta_tax, tip_amount, tolls_amount, improvement_surcharge, congestion_surcharge, total_amount)

        i. Stores the payment method and every charge for a trip
2. VENDOR, RATE_CODE, PAYMENT_TYPE, and LOCATION are each identified by their own id, which serves as a primary key. For the other tables:
    a. TRIP is identified by trip_id, and its vendor_id and ratecode_id are foreign keys that reference VENDOR and RATE_CODE; TRIP_STOP is identified by the composite key (trip_id, stop_type); its trip_id is a foreign key that references TRIP and its location_id is a foreign key that references LOCATION.
    b. PAYMENT is identified by payment_id. Its trip_id is a foreign key that references TRIP and is declared UNIQUE so that each trip has only one payment record, and it's payment_type_id is its foreign key that references PAYMENT_TYPE
3. Each table now contains only dependencies of its own primary key:
    a. For VENDOR, vendor_id determines vendor_name
    b. In RATE_CODE, ratecode_id determines rate_description
    c. PAYMENT_TYPE, payment_type_id determines payement_description
    d. LOCATION has no non-key attributes
    e. TRIP has trip_id determine the vendor_id, ratecode_id, passenger_count, trip_distance, and store_and_fwd_flag
    f. TRIP_STOP has the combination of trip_id and stop_type determines stop_datetime and location_id
    g. PAYMENT has payement_id determine trip_id, payment_type_id, and all eight attributes. Because trip_id is unique in PAYMENT, trip_id also determines payment_id. The attribute total_amount is kept as a stored value rather than treated as derived, because it equls the sum of other charges, so it cannot be reliably calculated from them.
4. Redundancy that was identified in the 1NF stage has been removed:
    a. Each trip's details and charges are now stored once instead of twice
    b. Each code's meaning is stored once in its own table rather than being implied across millions of rows.
    c. The only values that still repeat are foreign keys, i.e. the same vendor_id appearing in many trip rows.
5. No further changes were required
6. It is 2NF because every table in the final design is already in 1NF, and none of them contains a partial dependency:
    a. TRIP, PAYMENT, VENDOR, RATE_CODE, PAYMENT_TYPE, and LOCATION each have a single-attribute primary key, so no non-key attribute can depend on only part of a key.
    b. TRIP_STOP is the only table with a composite key. It's two nonkey attributes, stop_datetime and location_id, depend on the

whole key, because a stop's time and place depend on both the trip and whether the stop is the pickup or the dropoff.

PART 3



Part 4

- **VENDOR**

Attribute	Key	Description

vendor_id	Primary Key	Code identifying the vendor
vendor_name	None	Name of the technology company that recorded the trip

- **RATE_CODE**

Attribute	Key	Description
ratecode_id	Primary Key	Code identifying the fare rate type
rate_description	None	Name of he rate type the code stands for

- **PAYMENT_TYPE**

Attribute	Key	Description
payment_type_id	Primary Key	Code identifying how the passenger paid
payment_description	None	Name of the payment method the code stands for

- **LOCATION**

Attribute	Key	Description
location_id	Primary Key	Code identifying an NYC taxi zone

- **TRIP**

Attribute	Key	Description

trip_id	Primary Key	Surrogate identifier assigned to each trip
vendor_id	Foreign Key for **VENDOR** (vendor_id)	Vendor that recorded the trip
ratecode_id	Foreign Key for **RATE_CODE** (ratecode_id)	Rate type in effect at the end of the trip
passenger_count	None	Number of passengers, entered by the driver
trip_distance	None	Distance of the trip in miles
store_and_fwd_flag	None	Y if the trip record was stored in the vehicle and sent later; N if sent immediately

- **TRIP_STOP**

Attribute	Key	Description
trip_id	Primary Key, Foreign Key for **TRIP** (trip_id)	Trip that the sto belongs to
stop_type	Primary Key	Whether the stop is the pickup (P) or the dropoff (D)
stop_datettime	None	Date and time the meter was started (pickup) or stopped (drop-off)
location_id	Foreign Key for **LOCATION** (location_id)	Taxi zone where the stop took place

- **PAYMENT**

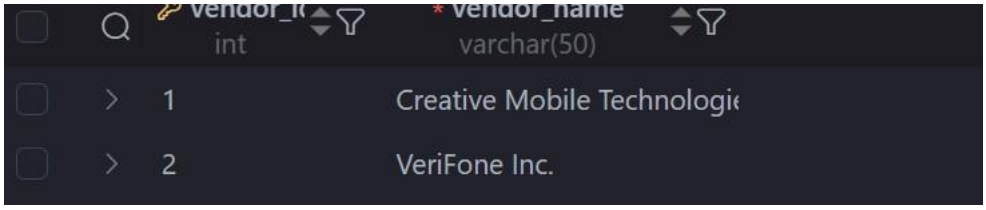
Attribute	Key	Description
payment_id	Primary Key	Surrogate identifier assigned to each payment record
trip_id	Foreign Key for **TRIP** (trip_id), **UNIQUE**	Trip that the payment belongs to
payment_id	Foreign Key for **PAYMENT_TYPE** (payment_type_id)	How the passenger paid
fare_amount	None	Time-and_distance fare calculated by the meter
extra	None	Extras and surcharges, such as rush-hour and overnight charges
mta_tax	None	$0.50 MTA tax
tip_amount	None	Tip amount (credit-card tips only)
tolls_amount	None	Total tolls paid during the trip
improvement_surcharge	None	$0.30 improvement surcharge
congestion_surchage	None	NYC congestion surcharge
total_amount	None	Total amount charged to the passenger, not including cash tips

Part 5 – Implementing the Database Using SQL

We created a single script for the database which we called “taxi_db” and the seven tables that we have created a 2NF: VENDOR, RATE_CODE, PAYMENT_TYPE, LOCATION, TRIP, TRIP_STOP, and PAYMENT. Every table name, column name, and primary key and foreign key in the script matches our relational schema from part 4 and the ER diagram.

The implementation can be found in our “create_tables.sql” file.

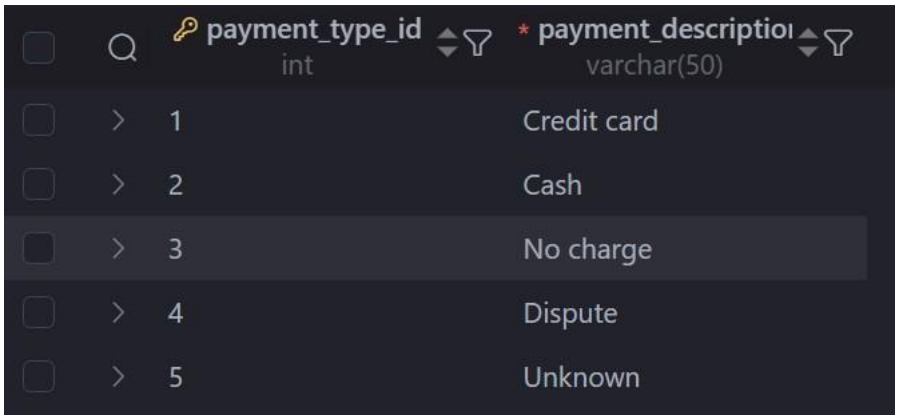
PART 6 AND 7 - SQL CODES AND DOCUMENTATION
VENDOR TABLE



RATE_CODE TABLE



PAYMENT_TYPE TABLE



LOCATION TABLE



location_id int
1
2
3
4
5
6
7
8
9
10

TRIP TABLE

trip_id int	vendor_id int	ratecode_id int	passenger_count int	* trip_distance decimal(10,2)	store_and_fwd_flag char(1)
1	1	1	1	1.20	N
2	1	1	1	1.20	N
3	1	1	1	0.60	N
4	1	1	1	0.80	N
5	2	1	1	0.00	N
6	2	1	1	0.03	N
7	2	1	1	0.00	N
8	2	5	1	0.00	N
9	2	1	4	0.00	N
10	1	1	2	0.70	N

TRIP _ STOP TABLE

trip_ int	stop_typ char(1)	* stop_datetime datetime	* location_id int
1	D	2020-01-01 00:33:03	239
1	P	2020-01-01 00:28:15	238
2	D	2020-01-01 00:43:04	238
2	P	2020-01-01 00:35:39	239
3	D	2020-01-01 00:53:52	238
3	P	2020-01-01 00:47:41	238
4	D	2020-01-01 01:00:14	151
4	P	2020-01-01 00:55:23	238
5	D	2020-01-01 00:04:16	193
5	P	2020-01-01 00:01:58	193
6	D	2020-01-01 00:10:37	193
6	P	2020-01-01 00:09:44	7

PAYMENT TABLE

payment_i int	* trip_i int	payment_type_id int	* fare_amount decimal(10,2)	* extra decimal(10,2)	* mta_tax decimal(10,2)	* tip_amount decimal(10,2)	* tolls_amount decimal(10,2)	* improvement_surch decimal(10,2)	* congestion_surcharg decimal(10,2)	* total_amount decimal(10,2)
1	1	1	6.00	3.00	0.50	1.47	0.00	0.30	2.50	11.27
2	2	1	7.00	3.00	0.50	1.50	0.00	0.30	2.50	12.30
3	3	1	6.00	3.00	0.50	1.00	0.00	0.30	2.50	10.80
4	4	1	5.50	0.50	0.50	1.36	0.00	0.30	0.00	8.16
5	5	2	3.50	0.50	0.50	0.00	0.00	0.30	0.00	4.80
6	6	2	2.50	0.50	0.50	0.00	0.00	0.30	0.00	3.80
7	7	1	2.50	0.50	0.50	0.01	0.00	0.30	0.00	3.81
8	8	1	0.00	0.00	0.00	0.00	0.00	0.30	2.50	2.81
9	9	1	2.50	0.50	0.50	0.00	0.00	0.30	2.50	6.30
10	10	1	8.00	3.00	0.50	2.35	0.00	0.30	2.50	14.15
11	11	1	12.00	3.00	0.50	1.75	0.00	0.30	2.50	17.55
12	12	2	9.50	3.00	0.50	0.00	0.00	0.30	2.50	13.30
13	13	1	17.00	3.00	0.50	4.15	0.00	0.30	2.50	24.95
14	14	1	6.00	0.50	0.50	1.96	0.00	0.30	2.50	11.76
15	15	1	28.50	0.50	0.50	4.84	0.00	0.30	2.50	37.14
16	16	2	9.00	3.00	0.50	0.00	0.00	0.30	2.50	12.80
17	17	1	4.00	3.00	0.50	1.00	0.00	0.30	2.50	8.80
18	18	2	11.50	3.00	0.50	0.00	0.00	0.30	2.50	15.30
19	19	2	24.50	0.50	0.50	0.00	0.00	0.30	0.00	25.80
20	20	4	3.00	3.00	0.50	0.00	0.00	0.30	2.50	6.80

View Related Row